

Лабораторна робота №4.

Нейронні мережі та згорткові архітектури на супутникових знімках

5 вересня 2026 р.

Мета

Реалізувати багат шаровий перцептрон двічі — власноруч на NumPy зі своїм зворотним поширенням і той самий у PyTorch — та довести їхню чисельну еквівалентність; далі побудувати згорткову мережу на супутникових знімках EuroSAT і виміряти, скільки насправді дає кожен інгредієнт сучасного рецепта: архітектура, ініціалізація, нормалізація, аугментація та перенесення навчання.

Теоретичні відомості

EuroSAT — 27 000 супутникових знімків 64×64 у десяти класах землекористування: посіви, ліс, промислова забудова, житлова забудова, річка, водойма та інші. Це прикладна задача дистанційного зондування, а не набір іграшкових об'єктів, і саме тому вона зручна: тут одразу видно, навіщо потрібне перенесення навчання. На відміну від рукописних цифр, клас визначається текстурою й кольором цілої сцени, тому повнозв'язна мережа на сирих пікселях помітно програє згортковій.

- **Зворотне поширення.** Градієнт за всіма параметрами рахується одним проходом назад за ланцюговим правилом, тому коштує приблизно стільки ж, скільки прямий прохід. Реалізацію завжди перевіряють скінченними різницями: помилка в градієнті виглядає як «модель погано навчається» і легко приймається за невдалий темп навчання.
- **Активації та зникаючий градієнт.** Похідна сигмоїди не перевищує $1/4$, тож у мережі з десяти шарів множник сягає 10^{-6} і перші шари майже не отримують сигналу. ReLU має похідну 1 на додатній півосі й цієї проблеми не створює, натомість може «вимикати» нейрони назавжди.
- **Згортковий шар.** Замість зв'язку кожного входу з кожним виходом застосовується невелике ядро в кожній позиції. Кількість ваг $C_{out}(C_{in}K^2 + 1)$ не залежить від розміру зображення, а спільні ваги дають еквіваріантність до зсуву. Для входу $32 \times 32 \times 3$ економія проти повнозв'язного шару — два порядки.
- **Регуляризація глибоких мереж.** Пакетна нормалізація стабілізує масштаби активацій усередині мережі, dropout заважає нейронам покладатися на конкретних сусідів, аугментація породжує нові приклади перетвореннями, що зберігають мітку. Який із трьох дає найбільший приріст саме на ваших даних — питання до виміру, а не до віри: на супутникових знімках набір допустимих перетворень ширший, ніж на звичайних фотографіях, і це може змінити відповідь.

Корисні посилання для ознайомлення:

- [EuroSAT](#) — опис набору й стаття авторів; вантажитися через `torchvision.datasets.EuroSAT(root, ...)` близько 94 МБ, файл 170 МБ — завантажуйте заздалегідь;
- [Stanford CS231n](#) — конспекти лекцій із поясненням кожного шару;
- [PyTorch · Transfer Learning Tutorial](#) — канонічний приклад донавчання попередньо навченої мережі;
- [Dive into Deep Learning](#) — підручник із виконуваним кодом на PyTorch;
- [PyTorch: Training a Classifier](#) — офіційний приклад саме на цьому наборі;
- [Google Colab](#) — середовище з GPU; на процесорі навчання займе десятки хвилин замість одиниць.

Порядок виконання

1. Завантажити EuroSAT, показати приклади зображень кожного класу й розподіл класів. Навести базову лінію — лінійну модель на сирих пікселях.
2. Реалізувати багатошаровий перцептрон на NumPy разом із власним зворотним поширенням, перевірити градієнт скінченними різницями та довести чисельну еквівалентність із тією самою мережею, зібраною засобами фреймворку.
3. Порівняти щонайменше чотири конфігурації «активація × ініціалізація» за однакових умов і пояснити, яка навчається найшвидше.
4. Спроектувати згорткову мережу, порахувавши кількість параметрів кожного шару вручну до запуску. Порівняти її з повнозв'язною за кількістю ваг, часом епохи й точністю.
5. Виміряти внесок пакетної нормалізації та dropout окремо й разом. Результати навести таблицею.
6. Порівняти помірну й надмірну аугментацію з навчанням без неї. Пояснити, які перетворення для цього набору зберігають мітку, а які — ні.
7. Побудувати криві навчання для двох розмірів тренувальної вибірки й матрицю помилок найкращої моделі. Знайти пару класів, які плутаються найчастіше, і пояснити чому.
8. Звести всі моделі в таблицю «параметри — час епохи — точність — приріст до базової лінії» та зробити висновок, який прийом дав найбільше.

Додаткові завдання (необов'язкові)

Ці пункти виходять за межі аудиторного часу й виконуються за бажанням; вони не впливають на основну оцінку, але зараховуються як бонус.

1. Розширити пункт 3 до повної сітки: чотири активації (`sigmoid`, `tanh`, `relu`, `gelu`) на двох ініціалізаціях — вісім конфігурацій на одному графіку.
2. Додати до згорткової мережі залишкові з'єднання (один блок `ResidualBlock`) і порівняти збіжність із мережею без них за однакової глибини.

3. Взяти `resnet18` із `torchvision.models` з вагами ImageNet, донавчити останній шар на EuroSAT і порівняти з власною мережею за точністю та часом навчання. Окремо виміряти ту саму `resnet18` з випадковими вагами: скільки епох потрібно, щоб вона наздогнала донавчену, і чи наздоганяє взагалі.

Контрольні запитання

1. Що станеться з багатошаровою мережею, якщо активацію замінити тотожною функцією?
2. Ви реалізували зворотне поширення, і модель не навчається. Як за одну перевірку відрізнити помилку в градієнті від невдалого темпу навчання?
3. Похідна сигмоїди не перевищує $1/4$. Яке число це дає для мережі з десяти шарів і що з цього випливає?
4. Скільки параметрів має згортковий шар із 64 фільтрами 3×3 над входом із 32 каналів, і чому це число не залежить від розміру зображення?
5. Чому на супутникових знімках різниця між повнозв'язною та згортковою мережею набагато більша, ніж на рукописних цифрах?
6. Для фотографій, знятих із землі, вертикальне віддзеркалення руйнує мітку, а для супутникових знімків — ні. Поясніть чому й назвіть, які ще перетворення EuroSAT допускає, а звичайні фотографії — ні.
7. Мережа, навчена з нуля, дала 75 %, а донавчена `resnet18` — 91 % за вдесятеро менший час. Поясніть, звідки береться ця різниця, і назвіть випадки, коли перенесення навчання не допоможе.